APUNTES

SEMANALES

Josué S. Hidalgo Sandoval - 2024800128



ANTERIORMENTE EN COMPILADORES...

JUEGO DEL PARSING

- Reglas del juego:
  - **E** → E A T | T
  - **A** → + | -
  - **T** → T | F
  - **F** → (E) | #
- Donde # significa que llegará algún número...



- BUEEENO... Imagínese las siguientes fichas del juego...

JUEGO DEL PARSING



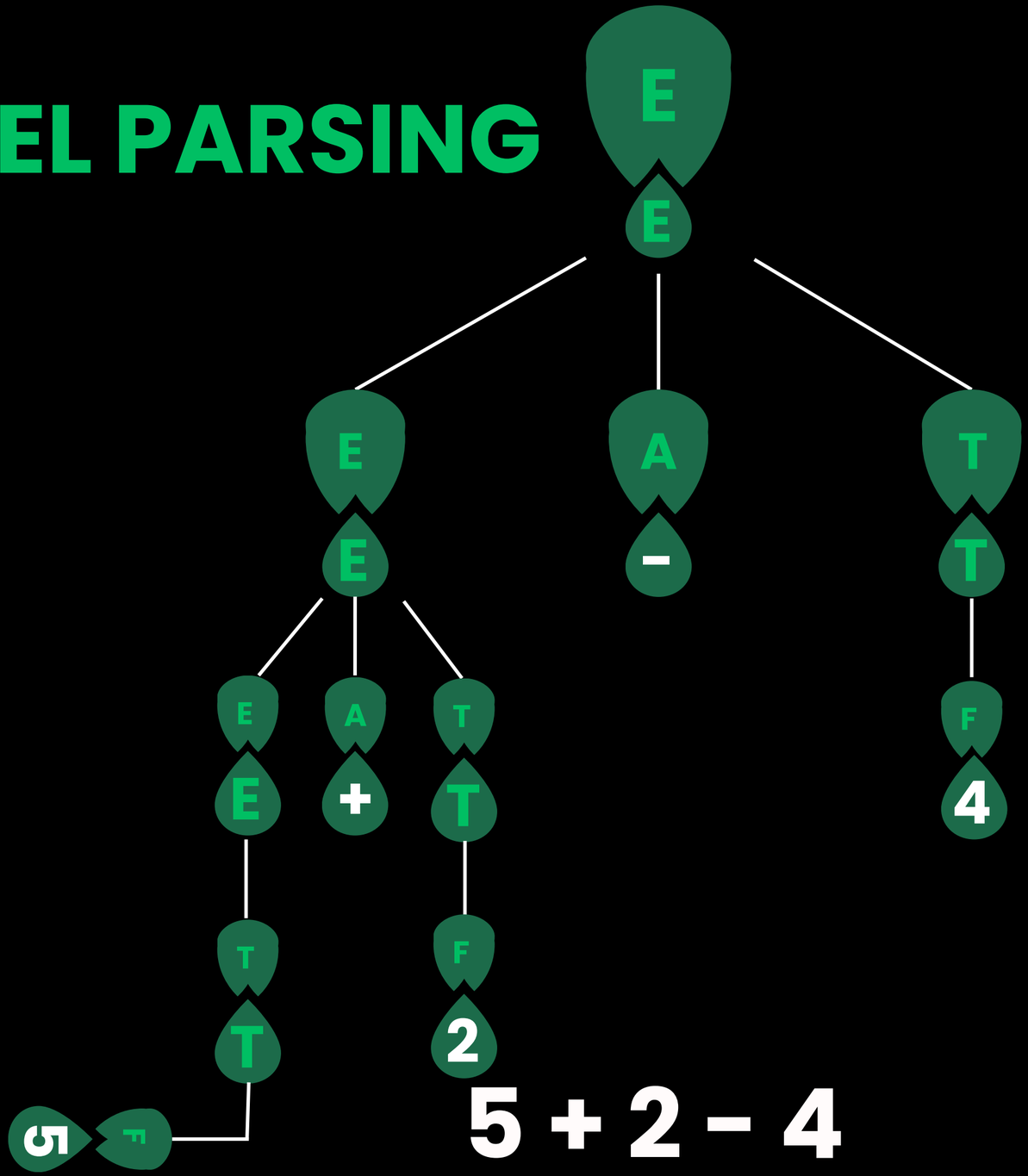
- Reglas del juego:
  - **E** → E A T | T
  - **A** → + | -
  - **T** → T | F
  - **F** → (E) | #

5 + 2 – 4



- Y el objetivo del juego es acomodar las fichas con letras blancas de tal forma que te quedes sin fichas.
- Puedes hacer uso de de las fichas verdes y las relaciones tienen que seguir las reglas del juego.

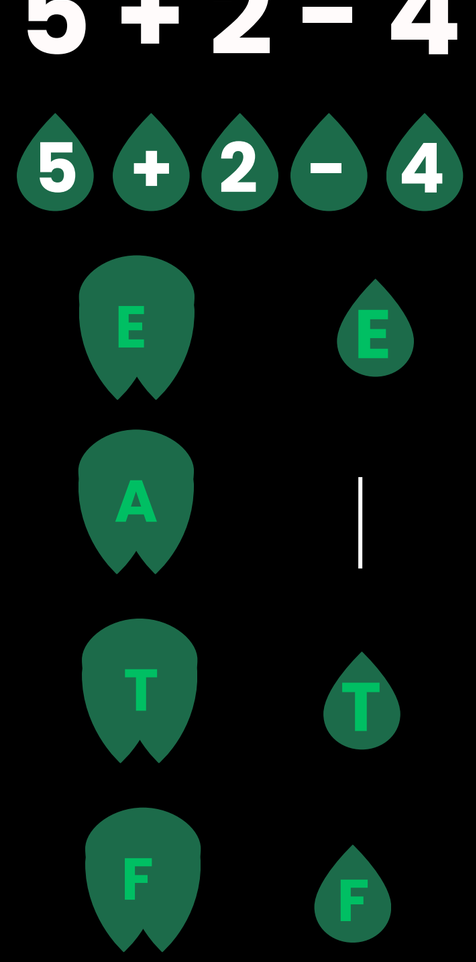
JUEGO DEL PARSING



5 + 2 – 4

JUEGO DEL PARSING

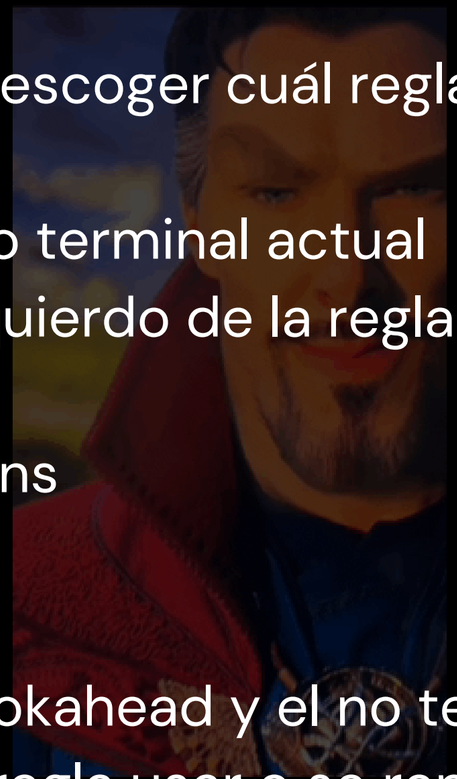
- ¿En qué dirección nos movemos?
  - Del símbolo inicial hacia la hilera:
    - TOPDOWN
  - De hilera hacia el símbolo inicial:
    - BOTTOMUP
- ¿Cómo sabemos cuál pieza usar?
  - Viendo el No Terminal abiuerto más izquierda.
  - Viendo el siguiente símbolo de la hilera (lookahead)
- ¿Podemos decidir con presición cuál regla usar?
  - Parsing Predictivo





PARSING PREDICTIVO

PREDICCIONES



- TOP DOWN
- En cada paso hay que escoger cuál regla usar
- Leftmost Derivation
- Siempre sabemos el no terminal actual
- Ya sabemos el lado izquierdo de la regla a usar puede haber muchas.
- Lookahead de “u” tokens

- **Parsing Predictivo**:
  - Con k tokens de lookahead y el no terminal actual se sabe exactamente cual regla usar o se reporta error de sintaxis.
  - k = 1

ELEMENTOS PARSING PREDICTIVO

CFG

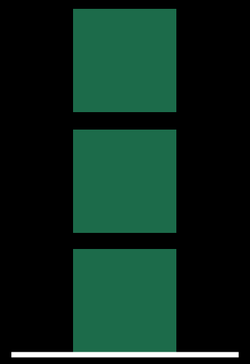
1. E → E A T | T
2. A → + | -
3. T → T M F | F
4. M → *
5. F → (F) | #

Hilera

Lookahead de 1

34 - 3 * 42

Pila



getToken()





Tabla de Acciones

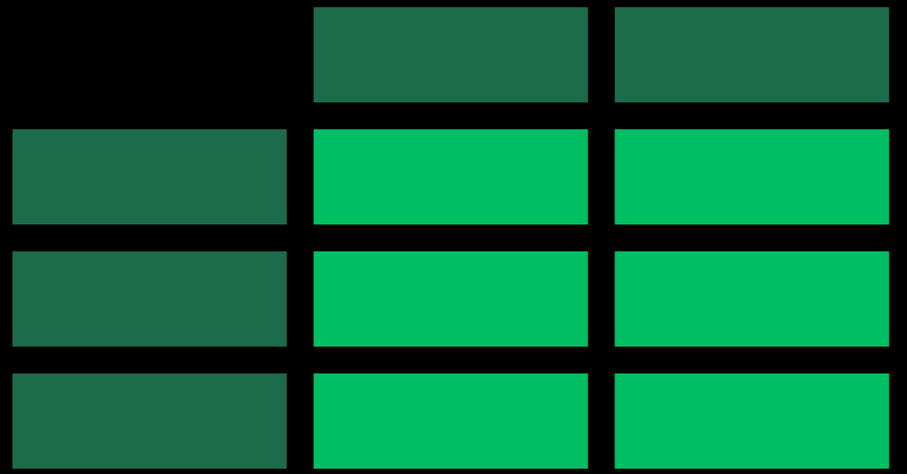
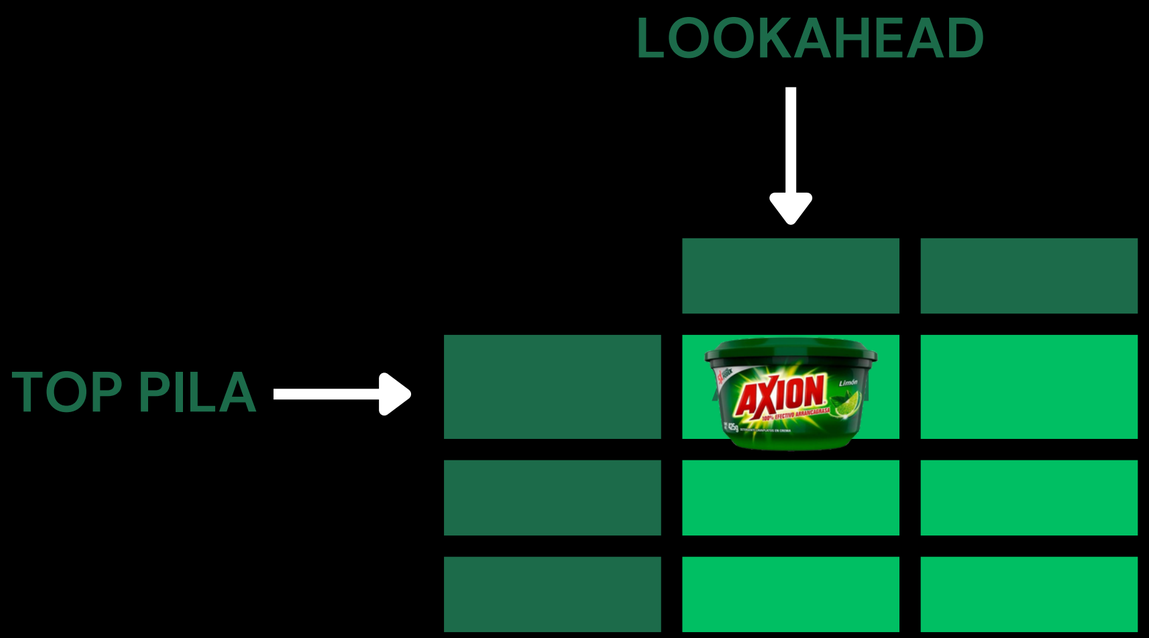


TABLA DE ACCIONES

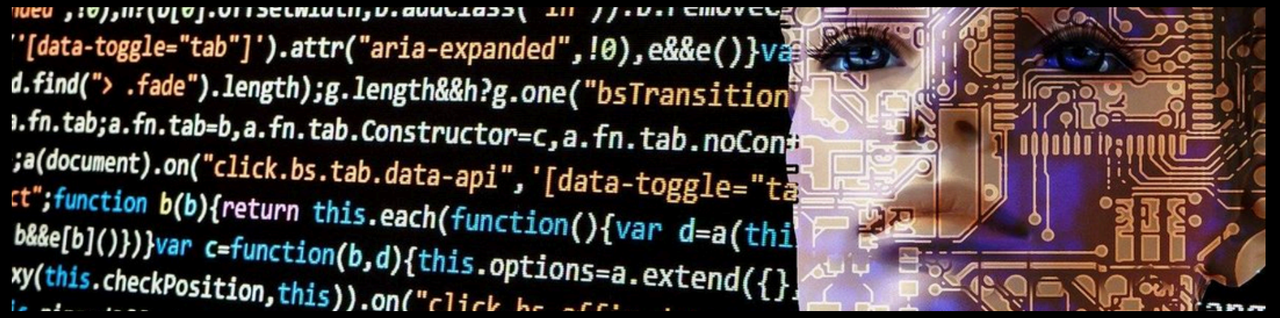
- Las **filas** subindexadas con el **top de pila**
- Las **columnas** están subindexadas con el **lookahead** actual
- En la **entrada** correspondiente de la tabla se indica la **acción a ejecutar**
- Hay algoritmos para generar esta tabla.



ALGORITMO DE PARSING PREDICTIVO

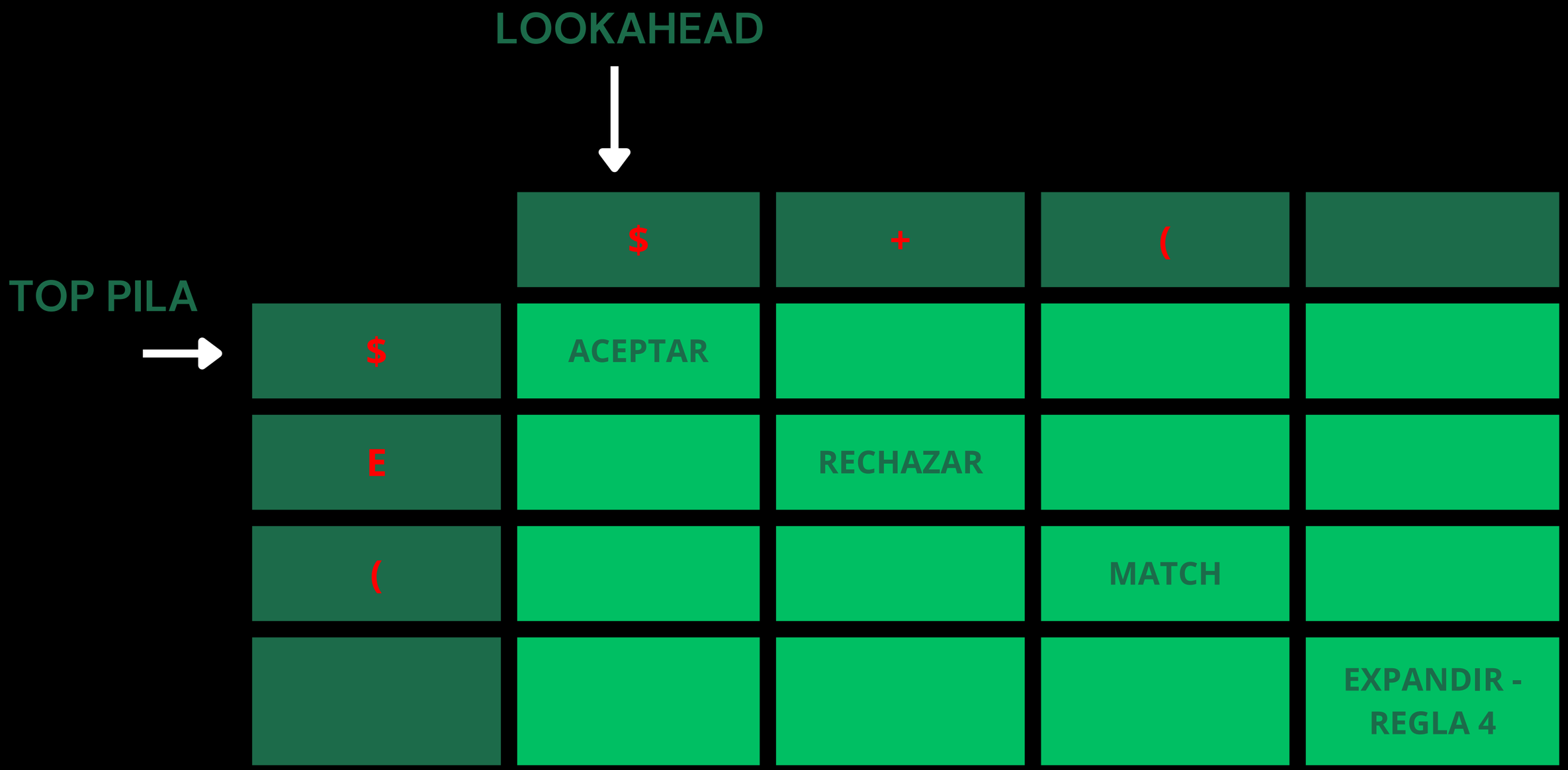
1. Agregar “$” (símbolo no miembro lenguaje) al final de la hilera a ser parseada.
2. Hacer un push en la pila vacía de “$”.
3. Hacer push de símbolo inicial de la gramática.
4. Establecer lookahead actual.
5. Mientras hilera no sea aceptada o rechazada.
    a. Realice acción en **Tabla [TOP][LOOKAHEAD]**

Ciclo



ACCIONES

- Las **entradas** de la Tabla de Acciones son **acciones**. 😌
- Hay varios tipos de acciones:
  - **ACEPTAR**
    - Aceptar hilera completa si fila y columna es $ e implica que la hilera es derivable. 😜
  - **RECHAZAR – ERROR**
    - Rechazar hilera completa si la combinación lookahead y top es inaceptable, a veces es entrada vacía. E implica que la hilera no es derivable con la CFG. 😔
  - **MATCH**
    - “Hacer match” si lookahead y top son el mismo token. Hay un terminal en el top de la pila.
    - Efecto: Avanzar en la entrada (**getToken()**). Y POP. 😄
  - **EXPANDIR**
    - Hay un no terminal en el top de la pila. Entrada contiene una regla o un número de regla.
    - Efectos: POP, PUSH del lado derecho de la regla, de tal manera que el símbolo más a la izquierda quede en el top de la pila (podría ser ε). 🧠


LOOKAHEAD
TOP PILA
$
+
(
$
ACEPTAR
E
RECHAZAR
(
MATCH
EXPANDIR - REGLA 4

CFG

1. S → (S)S
2. S → ε

TOP	LOOKAHEAD	ACCIÓN
$S	(()())$	

LOOKAHEAD

TOP

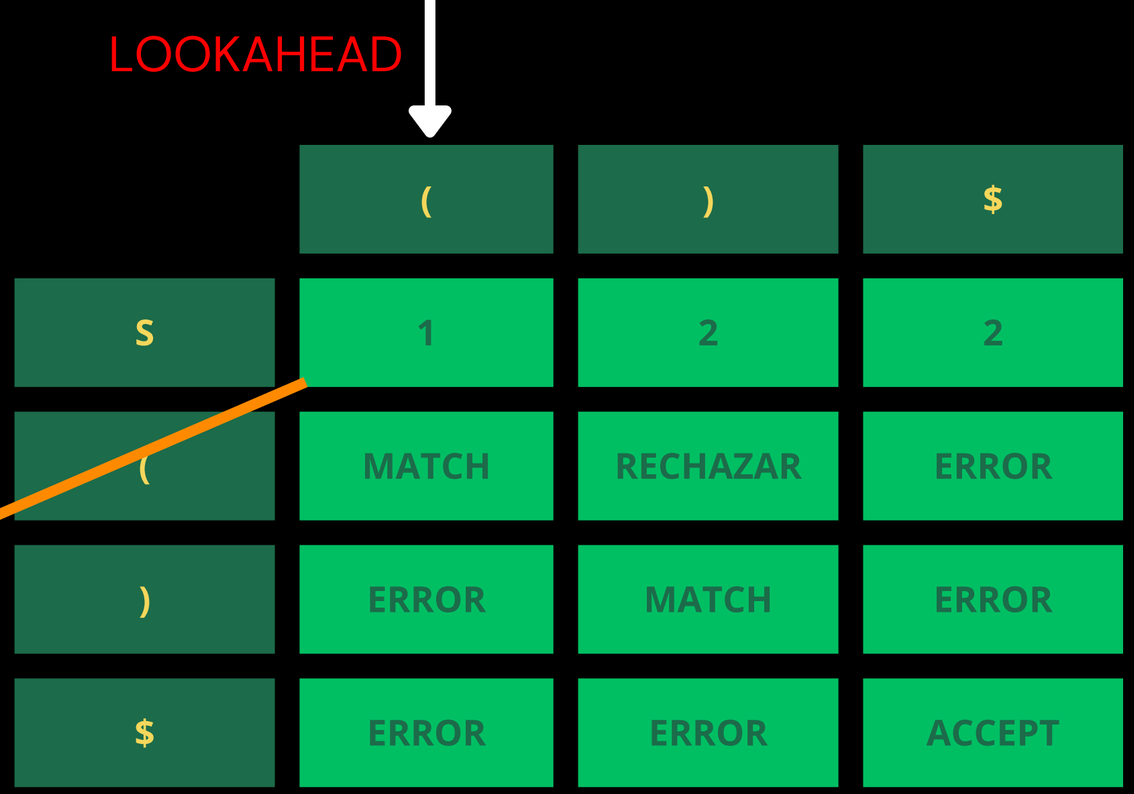
	(	)	$
S	1	2	2
(	MATCH	RECHAZAR	ERROR
)	ERROR	MATCH	ERROR
$	ERROR	ERROR	ACCEPT

1. Tome la entrada con Tabla[Lookahead][TopPila] y esa será la acción resultante de esos dos.

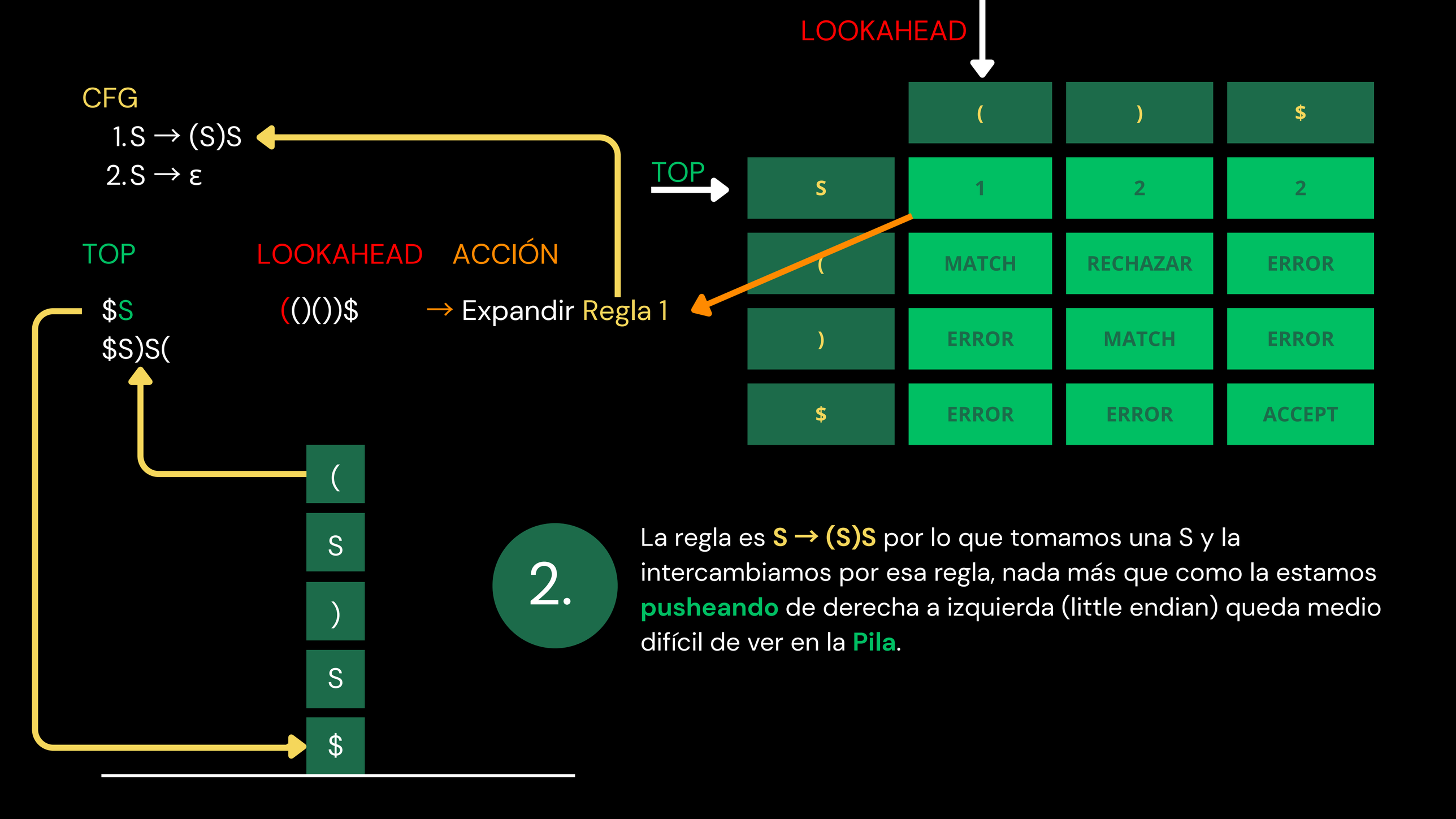
CFG

1. S → (S)S
2. S → ε

TOP	LOOKAHEAD	ACCIÓN
$S	(()())$	→ Expandir Regla 1



2. Note que la acción es un “EXPANDIR REGLA 1” y entonces la tendremos que expadir


LOOKAHEAD
CFG
1.S → (S)S
2.S → ε
TOP
LOOKAHEAD
ACCIÓN
$S
(()())$
→ Expandir Regla 1
$S)S(
TOP
(
)
$
S
1
2
2
(
MATCH
RECHAZAR
ERROR
)
ERROR
MATCH
ERROR
$
ERROR
ERROR
ACCEPT
(
S
)
S
$
2.
La regla es S → (S)S por lo que tomamos una S y la intercambiamos por esa regla, nada más que como la estamos pusheando de derecha a izquierda (little endian) queda medio difícil de ver en la Pila.

CFG

1. S → (S)S
2. S → ε

TOP	LOOKAHEAD	ACCIÓN
$S	(()())$	→ Expandir Regla 1
$S)S(	(()())$	→ Match
$S)S	((())$	

POP

getToken()

	(	)	$
S	1	2	2
(	MATCH	RECHAZAR	ERROR
)	ERROR	MATCH	ERROR
$	ERROR	ERROR	ACCEPT

LOOKAHEAD

TOP

3.

Cuando se hace “Match” entre el top y el lookahead entonces hacemos “getToken()” para avanzar con el lookahead y hacemos pop de top de la pila.

EJEMPLO 1

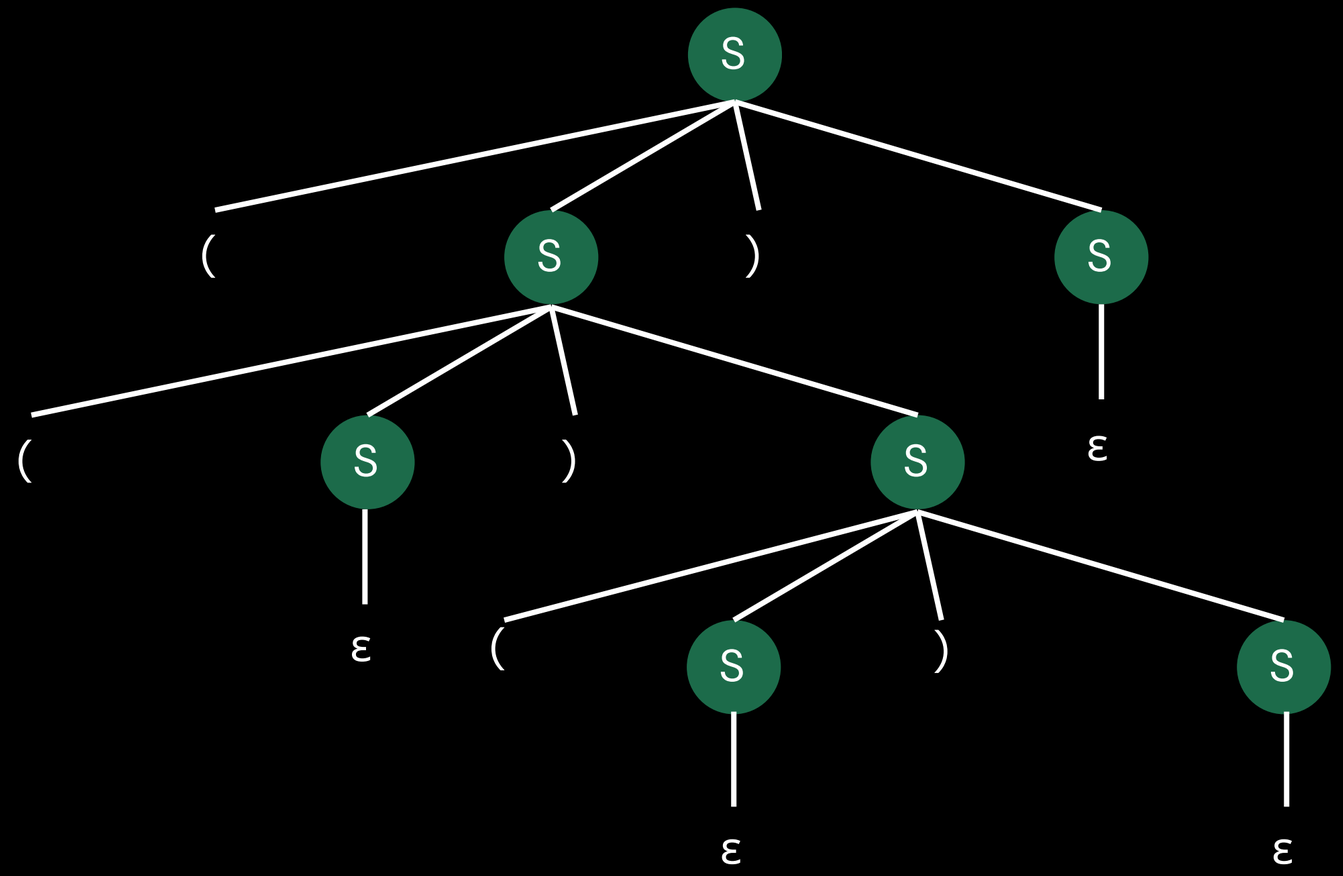
TOP	LOOKAHEAD	ACCIÓN
$S	(()())$	→ Expandir Regla 1
$S)S(	(()())$	→ Match
$S)S	(()())$	→ Expandir Regla 1
$S)S)S(	(()())$	→ Match
$S)S)S	(())())$	→ Expandir Regla 2
$S)S)	(())())$	→ Match
$S)S	(()(())$	→ Expandir Regla 1
$S)S)S(	(()(())$	→ Match
$S)S)S	(()()))$	→ Expandir Regla 2
$S)S)	(()()))$	→ Match
$S)S	(()())))$	→ Expandir Regla 2
$S)	(()())))$	→ Match
$S	(()()))$	→ Expandir Regla 2
$	(()()))$	→ ACCEPT

	(	)	$
S	1	2	2
(	MATCH	RECHAZAR	ERROR
)	ERROR	MATCH	ERROR
$	ERROR	ERROR	ACCEPT

CFG

1. S → (S)S
2. S → ε

POR CADA EXPAND




LA PIEDRA CUESTA C500

EJEMPLO 2

TOP	LOOKAHEAD	ACCIÓN
$S	()()$	→ Expandir Regla 1
$S)S(	()()$	→ Match
$S)S	)()$	→ Expandir Regla 2
$S)	)()$	→ Match
$S	()()$	→ Expandir Regla 1
$S)S(	()()$	→ Match
$S)S	)()$	→ Expandir Regla 2
$S)	)()$	→ Match
$S	()()$	→ Expandir Regla 2
$S)S(	()()$	→ ERROR

CFG

1. S → (S)S
2. S → ε

	(	)	$
S	1	2	2
(	MATCH	RECHAZAR	ERROR
)	ERROR	MATCH	ERROR
$	ERROR	ERROR	ACCEPT

EJEMPLO 3

TOP	LOOKAHEAD	ACCIÓN
$S	)()$	→ Expandir Regla 2
$	)()$	→ ERROR

CFG

1. S → (S)S
2. S → ε



	(	)	$
S	1	2	2
(	MATCH	RECHAZAR	ERROR
)	ERROR	MATCH	ERROR
$	ERROR	ERROR	ACCEPT

EJEMPLO 4

CFG

1. E → T E′
2. E′ → OP T E′
3. E′ → ε
4. OP → +
5. OP → –
6. T → F T′
7. T′ → M F T′
8. M → *
9. T′ → ε
10. F → (E)
11. F → #

	(	#	)	+	-	*	$
E	1	1					
E′			3	2	2		3
OP				4	5		
T	6	6					
T′			9	9	9	7	9
M						8	
F	10	11					

EJEMPLO 4

CFG

1. E → T E′
2. E′ → OP T E′
3. E′ → ε
4. OP → +
5. OP → -
6. T → F T′
7. T′ → M F T′
8. M → *
9. T′ → ε
10. F → (E)
11. F → #

TOP PILA	LOOKAHEAD	ACCIÓN
$E	3 + 4 $	→ Expandir con Regla 1
$E′ T	3 + 4 $	→ Expandir con Regla 6
$E′ T′ F	3 + 4 $	→ Expandir con Regla 11
$E′ T′ #	3 + 4 $	→ Match
$E′ T′	3 + 4 $	→ Expandir con Regla 9
$E′	3 + 4 $	→ Expandir con Regla 2
$E′ T OP	3 + 4 $	→ Expandir con Regla 4
$E′ T +	3 + 4 $	→ Match
$E′ T	3 + 4 $	→ Expandir con Regla 6
$E′ T′ F	3 + 4 $	→ Expandir con Regla 11
$E′ T′ #	3 + 4 $	→ Match
$E′ T′	3 + 4 $	→ Expandir con Regla 9
$E′	3 + 4 $	→ Expandir con Regla 3
$	3 + 4 $	→ ACCEPT



	(	#	)	+	-	*	$
E	1	1					
E′			3	2	2		3
OP				4	5		
T	6	6					
T′			9	9	9	7	9
M						8	
F	10	11					

¿DE DONDE VIENEN ESAS TABLAS MÁGICAS?

- Hay algoritmos para eso.
- Se requiere cierto preproceso de la Gramática.
- El lenguaje asociado no cambia.
  - Genera las mismas hileras



RECURSIVIDAD POR LA IZQUIERDA

- Una CFG es recursiva por la izquierda si hay al menos una regla de la forma:
  - $A \rightarrow A\alpha$
  - $A \rightarrow A\alpha \rightarrow A\alpha\alpha \rightarrow A\alpha\alpha\alpha \rightarrow A\alpha\alpha\alpha\alpha \rightarrow \ldots$
- Debe haber otra regla diferente que la detenga o aterrice:
  - $A \rightarrow \beta$
- Las hileras generadas son de la forma:
  - $\beta\alpha\alpha\alpha\alpha

¡¡¡ELIMINAR RECURSIVIDAD!!!

- **Tenemos** algo de la forma:
  - $A \rightarrow A\alpha$
  - $A \rightarrow \beta$
- Lo vamos a **cambiar por**:
  - $A \rightarrow \beta A'$
  - $A \rightarrow \alpha A'$
  - $A \rightarrow \varepsilon$

- Las hileras generalmente no cambian.
- Veámos un ejemplo.



EJEMPLO 1



1. E → T E′
2. E′ → +TE′
3. E′ → ε

EJEMPLO 2



OP → + | −



M → *
F → (E)
F → #

1. E → T E′
2. E′ → OP T E′
3. E′ → ε

---

4. OP → +
5. OP → −

---

6. T → F T′
7. T′ → M F T′
8. T′ → ε

---

9. M → *
10. F → (ε)
11. F → #

APUNTES

SEMANALES

Josué S. Hidalgo Sandoval - 2024800128

